TECHNICAL DOCUMENTATION

Receipt Analyser SaaS

AI-Powered Receipt & Expense Platform

Telegram Bot · Web Dashboard · Multi-Branch Business Layer

Node.js + Express · Supabase · Groq AI

Version 0.1.0 · Generated 2026-07-08

Internal — Team Reference

Table of Contents

01	Project Overview
02	System Architecture
03	Features
04	AI Workflow
05	Folder Structure
06	Database Overview
07	API Overview
08	Security
09	Deployment
10	User Guide
11	Future Improvements

01 Project Overview

Purpose

Receipt Analyser SaaS turns receipt/invoice photos into structured expense records automatically, via a Telegram bot or a web dashboard, using AI extraction.

Overview

Item	Detail
Product type	Vertical AI SaaS — expense capture + reporting
Primary input	Telegram photo/PDF, or dashboard upload
AI engine	Groq-hosted vision + text models
Backend	Node.js (≥20) + Express
Database	Supabase (PostgreSQL + Storage)
Frontend	Vanilla JavaScript (no framework)
Hosting	One isolated deployment per client
Tenancy	Single-account-per-instance

One codebase serves two modes via configuration, not a fork:

- **Personal** — one person tracking their own expenses.
- **Business** — multiple branches, staff, admin/clerk roles, petty cash, monthly statements.

Key Components

- **Telegram bot** — main capture channel; reads receipts, summarises daily/weekly totals.
- **AI pipeline** — vision model reads the image; rules + a text model categorise; results are validated.
- **Web dashboard** — list, filter, edit, review, report, export; business login for admin/clerk.
- **Business layer (M2)** — branches, staff, roles, petty cash, statement export, strict branch/company isolation.
- **Database** — Supabase Postgres holds every record; accessed only by the server (service-role key).

How It Works

1. A receipt arrives (bot or dashboard).
2. AI reads it into structured fields (merchant, date, amount, tax, category, line items).
3. Saved and tagged to the account (and, in business mode, branch/company/staff).
4. Users view/manage on the dashboard and export to Excel/PDF.

Notes

- Business features are flag-switched; off = personal product unchanged.
- Isolation is enforced at one application checkpoint today; database-level RLS is a planned hardening step.
- Anything not verifiable from code is marked **Requires Manual Review**.

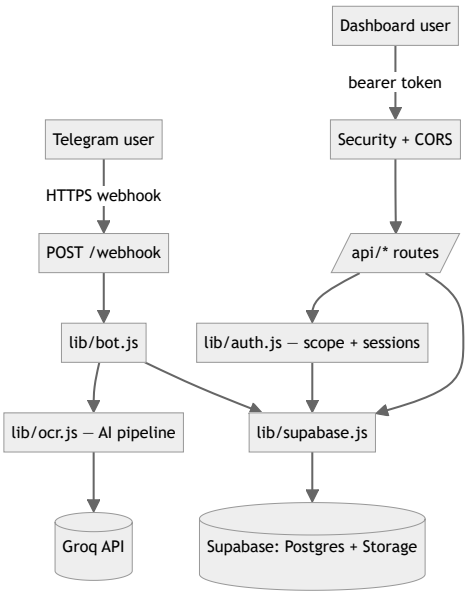
02 System Architecture

Purpose

How the parts fit together and how a request flows through the system.

Overview

A single **stateless Node.js process** (Express) with all state in Supabase — any always-on host works, and redeploys/rollbacks are safe. Two entry channels — the **Telegram webhook** and the **web dashboard/API** — meet at the same core services.



Key Components

Layer	Files	Responsibility
Entry/wiring	`server.js`	Mounts middleware + routes; starts server; registers webhook
Middleware	`security-headers.js` , `cors-preflight.js`	CSP/security headers; CORS preflight
Telegram	`webhook.js` , `bot.js` , `telegram.js`	Receive updates, run bot logic, send replies
AI pipeline	`ocr.js` , `groq.js` , `categorize.js` , `confidence.js`	Read receipts, extract, categorise, score
Web API	`routes/*.js`	Expenses, reports, review, agent, upload, business admin
Identity/isolation	`auth.js` , `supabase.js` , `account.js`	Login/sessions, scope checkpoint, capability flags
Business logic	`branches.js` , `pettyCash.js` , `statement*.js` , `reports.js`	Branches/staff, petty-cash maths, statements, reports
Data	Supabase	All persistent state

How It Works

Telegram flow: update hits `/webhook` (secret-gated) → `bot.js` resolves what it is (photo/PDF/album/menu/text) and, in business mode, the chat's binding → photos/PDFs go through `ocr.js` → Groq → saved via `supabase.js` → bot replies (and asks for staff tag in business mode).

Dashboard flow: browser calls `/api/*` with a bearer token → `checkDash` gate → in business mode the token resolves to a session → scope (`account_id` + role + branch/company) → `supabase.js` applies that scope to every query, so a clerk only ever sees their own branch → JSON returned, rendered by the vanilla-JS frontend.

Design Principles

- One codebase, capability flags — no personal/business fork.
- One isolation checkpoint — every business query goes through `scopeForUser` + `lib/supabase.js` .
- Stateless app, stateful database.
- Secrets stay server-side only.

Notes

- The bot keeps small, short-lived state in memory (e.g. "which receipt is being tagged"); a restart clears it harmlessly — records are always saved first.
- Model IDs are configuration; a model deprecation is a settings change.

03 Features

Purpose

What the product does today — shared core, personal, and business (multi-branch) layer.

Overview

Every item below exists in the current codebase. Business features activate only when their flag is on (`lib/account.js`); off = the personal product is unchanged.

Key Components

A. Core capture & AI (all modes)

Feature	What it does
Telegram photo capture	Send a photo → auto-read and logged
Multi-photo (album)	Several photos at once → all logged, one summary
PDF ingestion	Rendered to images, read the same way (size/pixel caps)
AI field extraction	Merchant, date, amount, subtotal, tax, currency, line items
Auto-categorisation	Rules + learned corrections + AI classifier
Confidence scoring	Low-confidence fields flagged; bot may ask to confirm
Category learning	A manual fix is remembered for next time
Currency handling	Stored in account default currency; original retained
Voucher number	Each expense gets `RC-2026-0041` (DB-generated, race-free)

B. Dashboard (web)

Feature	What it does
List + filters	Browse, search, filter saved expenses
Edit	Correct any field; category edits feed learning
Soft delete	Hidden, recoverable — never hard-deleted
Reports/Overview	Totals, category breakdowns, custom colours
Review queue	Admin approves/rejects flagged expenses
AI agent chat	Ask questions; native tool-calling over your data
Excel export	Formatted workbook + business branch/staff columns + receipt images
Mass upload	Upload many receipts at once
Template import	Import a CSV/Excel header template (your own columns/categories)

C. Telegram bot

Feature	Personal	Business
Today's Bills	List + per-bill ❌ delete	Staff name per line; one "Delete a bill → type number"
Today's Expenses	Today's total	Branch-scoped total
Weekly Record	Monday-start week	Sunday-start week
Category picker	Tap or "Others" + note	Same, plus the company's category set
Staff tagging	—	Pick who the receipt belongs to after saving

D. Business layer (M2 — flag-gated)

Feature	Flag	What it does
Multi-user roles	`multiUser`	Admin + clerk logins; staff as a name-only directory
Branches	`multiUser`	Partition an account into HQ + offices

Feature	Flag	What it does
Company dimension	`companyDimension`	Group → Company → Branch → Staff; per-company categories
Send to Admin	`branchSubmission`	Clerk submits a branch's month in-dashboard (audit-trailed)
Petty cash + claims	`pettyCash`	Monthly float + cash-in; balance/claims computed on read
Monthly statement	`pettyCash`	Excel + PDF statement per branch × month, receipts appended
Branch isolation	always (business)	Clerk sees own branch only; admin sees all

Notes

- Personal mode is a strict subset — business columns/tables exist but stay unused/null.
- First client's custom PDF statement layout: foundation built, exact match **Requires Manual Review** during delivery.

04 AI Workflow

Purpose

How the system turns a receipt image into structured, categorised data, and how the dashboard agent answers questions.

Overview

AI is provided by **Groq-hosted models** via an OpenAI-compatible chat API (`lib/groq.js`). Three jobs: **vision OCR**, **categorisation**, and the **dashboard agent**. Model choice is configuration (`lib/config.js → MODEL_ROUTES`), so a deprecation is a settings change, not code.

Job	Default model	Where
Vision (receipt reading)	`qwen/qwen3.6-27b`	`lib/ocr.js`
Text/agent	`llama-3.3-70b-versatile`	`lib/agent.js`
Classify (category)	`openai/gpt-oss-20b`	`lib/categorize.js`

Key Components

- **`groq.js`** — shared request runner; API-key pool rotation + backoff on rate limits.
- **`ocr.js`** — prepare image → vision model → parse → validate → assemble expense.
- **`categorize.js`** + **`categoryMemory.js`** — rules, learned corrections, AI classifier.
- **`confidence.js`** — marks fields high/low confidence.
- **`agent.js`** + **`agentProposals.js`** — dashboard chat agent + tool-calling loop.

How It Works



1. Image (or each PDF page) sent to the vision model with an extraction prompt.
2. `ocr.js` parses and validates the reply (dates kept flexible so bad OCR never crashes a save).
3. Category decided in order: **rules** → **learned corrections** → **AI classifier**.
4. Each field gets a confidence label; low-confidence key fields can trigger a bot follow-up.
5. Record saved; business mode later adds a staff tag.

Dashboard agent: the user asks a question (`POST /api/agent`) → the model is given tools to query expense data → it calls them, reads results, replies in plain language. Any proposed action is confirmed separately (`/api/agent/confirm`) before anything changes.

Reliability & Notes

- **Key pooling** — multiple Groq keys rotate on rate-limit with no delay; single key = simple retry/backoff.
- **Abuse guards** — PDF byte-size/pixel caps checked **before** any AI call.
- **Graceful degradation** — categorisation still works (rules + AI) if learning data is absent.
- Raw model output is stored per expense for debugging/audit.
- **Data privacy:** regulated clients get the AI provider's **Zero-Data-Retention** mode; provider swap is a config point. Enabling it live is an operational step — **Requires Manual Review** per instance.

05 Folder Structure

Purpose

How the codebase is organised, for fast navigation.

Overview

Clean split: `server.js` wires everything, `lib/` holds logic, `routes/` holds HTTP endpoints, `public/` is the dashboard, `docs/` is project documentation/decisions.

```
receipt-analyser-saas/
├── server.js                # Entry point — mounts middleware + routes
├── schema.sql               # Full database schema (run once on Supabase)
├── package.json              # Dependencies + scripts
├── render.yaml / Dockerfile / fly.toml   # Deployment configs
│
├── lib/                     # All logic (36 modules)
│   ├── config.js / account.js         # Env config, model routing, capability flags
│   ├── auth.js / supabase.js          # Sessions, scopeForUser, DB access + isolation
│   ├── bot.js / telegram.js           # Bot conversation logic + Telegram API client
│   ├── ocr.js / groq.js / groqkeys.js # AI pipeline + provider client + key rotation
│   ├── categorize.js / categoryMemory.js / confidence.js
│   ├── agent.js / agentProposals.js   # Dashboard AI agent
│   ├── branches.js / pettyCash.js     # Branch/staff, petty-cash maths (business)
│   ├── statement.js / statementXlsx.js / statementPdf.js
│   ├── reports.js / excel.js / templates.js
│   ├── pdf.js / storage.js            # PDF rendering, receipt image storage
│   ├── anomaly.js / reviewDecision.js / expenseValidation.js
│   ├── money.js / currency.js / ratelimit.js
│   ├── security-headers.js / cors-preflight.js / log.js / http.js
│
├── routes/                  # HTTP endpoints (17 files)
│   ├── webhook.js / auth.js
│   ├── expenses.js / receipt.js / reports.js / excel.js / review.js
│   ├── agent.js / upload.js / templates.js / settings.js / misc.js
│   └── branches.js / companies.js / submissions.js / pettyCash.js / statement.js
│
├── public/                  # Browser dashboard (vanilla JS, no framework)
│   ├── index.html / css/app.css
│   └── js/  (app.js, api.js, render.js, authgate.js + components/)
│
├── setup/                   # Admin provisioning CLI (setup-admin)
├── tests/                   # Automated tests (node --test) — 100+ files
└── docs/                    # Architecture, decisions, specs, runbooks, status
```

Key Components

Folder	Contains	User-facing?
`lib/`	All logic — testable, framework-free	No
`routes/`	Thin HTTP handlers calling `lib/`	No
`public/`	The dashboard	Yes
`docs/`	Architecture, ADRs, runbooks	Team
`tests/`	Automated tests	Team

Notes

- Routes stay thin — real logic lives in `lib/`; `server.js` is wiring only.
- Frontend uses **external** JS/CSS only (no inline scripts) to satisfy a strict CSP.

06 Database Overview

Purpose

The data model: what's stored, how tables relate, and how tenant/branch isolation works.

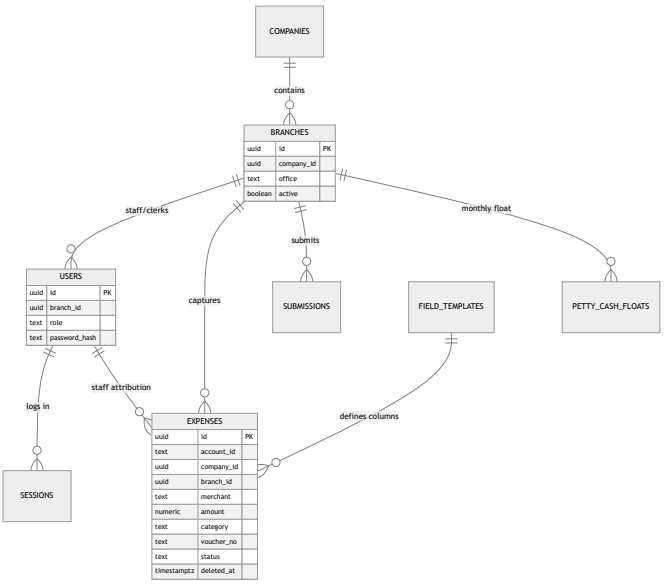
Overview

Database is **Supabase (PostgreSQL)**; full schema in `schema.sql`, applied once per client. Every row carries `account_id` (one account = one client instance). Business tables add `company_id` / `branch_id` for finer partitioning. The server connects with the **service-role key**; RLS is **enabled on every table with no public policy** (anonymous access denied).

Key Components (tables)

Table	Purpose
`expenses`	Core record — one row per captured receipt
`field_templates`	Per-account custom columns + category set
`category_corrections`	Learned category fixes
`voucher_counters`	Per-account/year counter behind voucher numbers
`account_settings`	Per-account UI preferences
`companies`	(Business) top level — a legal entity
`branches`	(Business) HQ + offices
`users`	(Business) admin/clerk logins + staff directory
`sessions`	(Business) opaque sessions (hash only stored)
`bot_bindings`	(Business) links a Telegram chat to a user/branch/role
`submissions`	(Business) "Send to Admin" per branch × month
`petty_cash_floats / cash_in`	(Business) monthly float + top-ups

Relationships



How It Works

- **Tenant key:** every table filtered by `account_id` — one instance = one client.
- **Business partitioning:** `branch_id` / `company_id` narrow a clerk to their slice; null for personal accounts.
- **Soft delete:** expenses hidden via `deleted_at`, never physically removed.
- **DB-generated values:** voucher numbers and `updated_at` set by database triggers, so every write path gets them automatically. An atomic upsert powers race-free category learning.

- **Computed, not stored:** petty-cash claim status is calculated on read, so it can't drift from the numbers.

Notes

- Isolation today = application layer (`scopeForUser` checkpoint + service-role key); JWT-scoped RLS is a **later hardening step** (commented in `schema.sql`).
- Business tables were added additively — personal data was never migrated or disturbed.
- Legacy rows before the company dimension have `company_id = null` ; queries are null-tolerant so they stay visible.

07 API Overview

Purpose

The HTTP endpoints, who can call them, and how requests are secured.

Overview

All endpoints are Express routes mounted in `server.js`. Data endpoints live under `/api/*` and require a bearer token (`checkDash`). Business endpoints additionally require the matching capability flag and role. The Telegram webhook is a separate, secret-gated endpoint.

Key Components (endpoints)

Public / system

Method	Path	Purpose	Auth
GET	`/`	Serve dashboard	none
GET	`/health`	Liveness check	none
GET	`/ready`	Deep readiness probe	rate-limited
POST	`/webhook`	Telegram updates	secret header
GET	`/api/config`	Public front-end config	none

Expenses & receipts

Method	Path	Purpose
GET	`/api/expenses`	List (branch-scoped for a clerk)
PATCH / DELETE	`/api/expenses/:id`	Edit / soft-delete
PATCH	`/api/expenses/payments`	Bulk mark-paid (admin)
GET	`/receipt/:fileId`, `/receipt/by-id/:id`	Fetch a receipt image

Reporting, export, review, agent

Method	Path	Purpose
GET	`/api/reports`	Aggregated report data
GET	`/api/excel`	Excel export
GET / POST	`/api/review`, `/api/review/:id`	Review queue list / decide
POST	`/api/agent`, `/api/agent/confirm`	Ask the AI agent / confirm its proposed action
POST	`/api/upload`	Mass-upload from dashboard
GET / POST	`/api/templates`	Read/import a custom field template
GET / PUT	`/api/settings/category-colors`	Read/save donut colours

Business (flag + role gated)

Method	Path	Purpose	Gate
POST	`/api/login`, `/api/logout`	Business session	multiUser
GET/POST/DELETE	`/api/branches[/:id]`, `/…/staff`	Manage branches/staff	admin
GET/POST/DELETE	`/api/companies[/:id]`	Manage companies	companyDimension, admin
GET/POST/PATCH	`/api/submissions[/:id]`	Send-to-Admin flow	branchSubmission
GET/PUT/POST/DELETE	`/api/petty-cash*`	Floats + cash-in	pettyCash, admin
GET	`/api/statement`	Monthly statement (Excel/PDF)	pettyCash

How It Works

1. `checkDash` gates every `/api/*` and `/receipt` route — no valid bearer token, no data; **fails closed** in production.

2. In business mode the token resolves to a **session** → **scope** (`account_id` , role, branch/company). `lib/supabase.js` applies that scope to every query and stamps it on every insert.
3. Business routes also check the **capability flag** and **role**; non-qualifying requests get a 404/403.
4. **Errors are generic** — a global handler returns `{"error": "…"}` , never a stack trace or file path.

Notes

- Request bodies have per-route size limits (e.g. 16 KB login, 10 MB upload).
- Sensitive routes carry their own rate limiters (login, agent, upload, statement, admin).
- Exact field shapes per endpoint live in the route files/tests — treat those as the contract.

08 Security

Purpose

Authentication, isolation, data protection, and hardening controls.

Overview

Layered security: secrets stay server-side, every data route is token-gated, business data is isolated at one enforced checkpoint, and every response carries strict security headers.

Key Components

Authentication & sessions

- Dashboard bearer token (`DASHBOARD_TOKEN`) gates every `/api/*` + `/receipt` route; **fails closed** in production if unconfigured.
- Business login: passwords hashed with **scrypt** (memory-hard, Node core, no extra dependency).
- Sessions are opaque random tokens; only the **SHA-256 hash** is stored — a leaked DB dump can't be replayed. Default 12h expiry, revocable per user.

Isolation (multi-branch/company)

- One checkpoint: `scopeForUser(user)` → `{account_id, role, branch/company}` . `lib/supabase.js` applies it to **every** query and stamps it on **every** insert — no route hand-rolls its own filter.
- Clerk = hard-filtered to own branch/company. Admin = sees all by design. Staff never log in.
- Company filter is **null-tolerant** — legacy rows stay visible, cross-company reads still blocked.
- Independent isolation audits + two-direction automated tests are standing practice.

Data protection

- Secrets server-side only — browser never talks to Supabase/AI provider directly.
- Service-role key used by the server; RLS enabled with **no anonymous policy** anywhere.
- No PII in logs — structured error logs record message + scalar route context only.
- Soft delete keeps financial records recoverable and auditable.

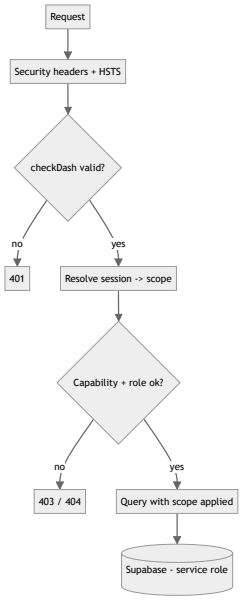
HTTP hardening (`lib/security-headers.js`)

Header	Effect
Content-Security-Policy	Same-origin scripts/styles only, no inline
X-Content-Type-Options / X-Frame-Options	No MIME sniffing / no clickjacking
Referrer-Policy / Permissions-Policy	No referrer leak; camera/mic/geo disabled
Cross-Origin-Resource-Policy	Others can't read our responses
Strict-Transport-Security	Enforced on HTTPS
Cache-Control: no-store	On `/api/*` and `/receipt`

Input & abuse controls

- Webhook secret required for Telegram updates; chat allowlist covers button taps too.
- Per-route body-size limits + rate limiters (login, agent, upload, statement, admin).
- PDF/upload byte-size + rendered-pixel caps checked **before** any AI call.
- Generic errors only; no raw SQL (PostgREST + encoded params).

How It Works



Notes — pre-launch gates (before real customers)

Rotate all dev-used secrets · enable AI provider Zero-Data-Retention for regulated clients · hold an independent DB backup · return demo-relaxed rate limits to production values. **Future hardening (recorded, not yet enabled):** JWT-scoped RLS at the DB layer; shorter session expiry/auto-logout.

09 Deployment

Purpose

How a client instance goes from repository to a live, always-on, secure service.

Overview

A single **stateless Node.js (≥20) process** — all state in Supabase, so any always-on host works and rollbacks are safe. Model: **one small deployment per client**. Two supported hosts: **Render** (native Node, default) and **Fly.io** (Docker, portable). Two hard requirements: **HTTPS** (Telegram requires it) and **always-on** (no free/scale-to-zero plans).

Key Components

Item	Role
`render.yaml`	Render blueprint (build/start, health check, region)
`Dockerfile` + `fly.toml`	Fly.io container + always-on config
Environment variables	Secrets + capability profile (never committed)
Supabase project	Data plane — schema applied, first admin created
`npm run setup-admin`	One-time CLI to mint the first business admin
`npm run setup-webhook`	Registers the Telegram webhook against the host URL

Required environment variables

Var	What
`BOT_TOKEN`	Telegram bot token
`GROQ_KEY` / `GROQ_KEYS`	AI provider key(s) — comma list enables rotation
`SB_URL` / `SB_KEY`	Supabase URL + service-role key
`WEBHOOK_SECRET`	Random 32+ chars — mandatory in production
`DASHBOARD_TOKEN`	Random 32+ chars — dashboard login
`AUTH_REQUIRED`	`true` — fail-closed auth
`PUBLIC_URL`	Deployed HTTPS URL — set **after** first deploy
`ACCOUNT_ID` , `ACCOUNT_TYPE` , `CAP_*`	Per-client id + capability profile

How It Works

1. **Pre-flight:** Supabase provisioned (schema + first admin); secrets in hand.
2. **First deploy:** builds and boots (webhook registration skipped until `PUBLIC_URL` set — expected).
3. **Set** `PUBLIC_URL` to the live HTTPS URL, redeploy → app auto-registers the Telegram webhook.
4. **Verify (all four):** `/health` ok → `getWebhookInfo` shows the URL, no errors → send a receipt end-to-end → log in to the dashboard and see it.

Notes

- CI ≠ CD — tests run on every change; deploys are deliberate and on-demand.
- Rotating the bot token breaks live receipt thumbnails — re-register the webhook after any change.
- Rollback is safe on both hosts (stateless app; Supabase untouched).
- Uptime monitoring on `/health` and log aggregation are operational follow-ups — **Requires Manual Review**.

10 User Guide

Purpose

A simple, step-by-step guide for everyday users — no technical knowledge needed.

Overview

Two ways to use the system: the **Telegram bot** (fastest — just send a photo) and the **web dashboard** (viewing, reporting, exporting). Business users have roles: **admin** (sees everything) and **clerk** (sees their own branch).

Part 1 — Telegram bot

Log a receipt: send a photo → the bot reads it and replies with the details. If unsure of the category, it asks you to pick (or "Others" + a short note). Send **several photos at once**, or a **PDF** — same result.

Menu button	What you get
📋 Today Bills	Everything captured today
💰 Today Expenses	Today's total
🗓️ Weekly Record	This week's total
🧭 Bot Guide	Quick help

Delete a bill: Personal — tap ❌ next to it in Today Bills. Business — tap 🗑️ **Delete a bill**, then reply with the bill's number (only your own bills).

Business extra: after sending a receipt, tap which staff member it belongs to; Today Bills then shows the staff name on each line.

Part 2 — Web dashboard

Log in: Personal — access token. Business — username + password.

Action	How
See expenses	Listed on the main screen; filters to narrow down
Fix a detail	Click and edit (a category fix is remembered)
Remove one	Delete — hidden but recoverable
Totals & charts	Overview/Reports section
Ask a question	AI assistant, e.g. "How much on fuel this month?"
Export	Excel download
Bulk add	Upload area for many receipts at once

Business admin extras

Action	Who	What
Add branches & staff	Admin	Set up offices and people
Review flagged bills	Admin	Approve or reject
Set monthly petty-cash float	Admin	Opening cash per branch
Send to Admin	Clerk	Submit branch's month
Monthly statement	Admin	Download petty-cash statement (Excel/PDF) with receipts

Notes

- Records are saved before any summary is shown — a quiet bot doesn't mean lost data.
- Dates use Malaysia time (UTC+8).

11 Future Improvements

Purpose

Planned improvements and hardening steps — based on the code, decision log, and open issues, not speculation.

Overview

The core product (personal + business) is built and tested. Remaining work falls into three groups: **pre-launch operations**, **security hardening**, and **product enhancements**.

Key Components

A. Pre-launch operations

Item	Notes
Rotate all secrets	Dashboard token, AI keys, DB key used during development
Enable AI Zero-Data-Retention	On the business client's AI account
First validated production deploy	Runbook exists; a verified end-to-end deploy is the gate
Uptime monitoring	External monitor on `/health`
Independent DB backup	Held outside the primary account
Reset demo rate limits	Return relaxed limits to production values

B. Security hardening (recorded, not yet enabled)

Item	Notes
Database-level RLS	JWT-scoped policies beneath the app-layer scope
Session expiry tuning	Shorter, rotating sessions for business users
Legacy `company_id` backfill	Needed before a "company-only" role
Template label sanitisation	Extra defence-in-depth on imported labels

C. Product enhancements

Item	Notes
Client-specific statement layout	Faithful reproduction of first client's format
Per-image receipt links (batch)	Keep each photo's own link in a multi-photo batch
Bot help text polish	Align guide text with the newest delete flow
"Edit in place" on bot delete	Update the list message instead of a fresh one
Observability Phase 2	Optional error tracking, default off, per-client opt-in
Multi-tenant evolution	Shared multi-tenant + RLS if scale demands

How It Works (prioritisation)



Notes

- Exact scope/timing of each item is a business decision — **Requires Manual Review**.
- Core promise unchanged: personal stays as-is; business features are additive and flag-gated.